Day 2 Session summary- ASDXDDL8M2 Fundamentals of SQL

Thursday, 15th October 2026

Overview

Key participants: the QA Trainer (session lead) and learners including Melinda, David, Julia, Lindsay, Crawford, Emily, Ewelina, Victoria, Kim, Chris, Megan and others.

This two-day session reinforces day-one fundamentals and advances learners through reusable objects and advanced grouping. The QA Trainer recaps core SELECT/WHERE/ORDER BY patterns and then shows how to persist repeated logic in views, stored procedures and table-like constructs (table-valued functions, derived tables, temporary tables). He demonstrates creating and editing views, authoring parameterized stored procedures, and explains execution and parameter requirements (noting callers must supply required parameters), illustrating how these objects simplify reporting and permissions.

The trainer introduces advanced aggregation tools—ROLLUP and CUBE—to generate hierarchical subtotals and combinational groupings for multi-level reports, and he contrasts their behavior. He also clarifies transaction semantics (completed vs not completed) to provide basic context for data integrity. Throughout, the trainer troubleshoots lab issues (CONVERT/ISNULL), offers to join learner VMs for debugging, and explains lab mechanics including the shift to QATSQL script templates and file-persistence options (OneDrive/Google Drive/GitHub vs GoToMyPC native transfer).

Decisions and commitments: the QA Trainer commits to continued lab support including joining individual VMs on request and to help configure QA portal entries for learners (as done for Julia). He instructs learners to mark portal content complete to download the certificate and to upload the certificate plus a reflection statement for module sign-off.

Concerns and guidance: the trainer cautions that temporary tables are session-scoped, stored procedures require callers to provide parameters, and advanced labs (modules 9–11) are complex so learners should use solutions as needed. The trainer recommends practicing with provided script templates and external resources to solidify skills and ensures availability for follow-up questions about portal access and certification.

Detailed Summary

Day-two revision activity

The trainer leads a 30-second recall activity to review key concepts from day one, reinforcing retention and preparing learners for new material. Participants list topics they remember (relational databases, SQL clauses, operators, functions), and the trainer uses responses to confirm understanding and identify areas to expand on. The class uses the activity to transition into more advanced topics on day two.

- QA Trainer explains the morning revision exercise and instructs learners to post five concepts from day one in chat.
- Melinda and David reference relational databases and spreadsheet vs database differences as recalled topics, as noted by the QA Trainer.

- Julia, Lindsay, and others recall SQL statements and clauses such as SELECT, FROM, WHERE, ORDER BY, and logical operators, as reiterated by the QA Trainer.

SQL operators and phrasing

The trainer reviews logical, comparison, and string operators and advises preferring positive/assertive query phrasing rather than NOT-based negatives. The discussion clarifies usage of NOT (including NOT IN, NOT LIKE) and emphasizes writing clear, explicit filters to reduce ambiguity in query intent. The trainer notes functions and casting will be covered later to handle cases where type conversion or NULL handling affects operator use.

- Crawford asks why using NOT is discouraged, prompting the QA Trainer to explain framing queries positively when possible.
- QA Trainer clarifies that NOT is valid but can produce less direct intent and often interacts with operators like NOT IN and NOT LIKE.
- QA Trainer advises learners to ask precisely for what they want (e.g., 'age 24') to improve clarity.
- QA Trainer notes that functions and type-conversion (covered later) can affect how comparisons and NULL checks behave.

Aggregate functions and select usage

The class revisits aggregate functions, expression and filtering operators, concatenation, and use of brackets; the trainer reminds learners which functions appear in SELECT versus WHERE and how aggregates interact with GROUP BY and ORDER BY. The trainer highlights how these constructs affect query outcomes and previews that today's lessons extend these topics to multi-table contexts.

- QA Trainer summarizes that many functions and aggregates appear in the SELECT clause and influence GROUP BY and ORDER BY usage.
- Chris and Emily's points about concatenating columns and simple arithmetic operations are reiterated by the QA Trainer to show string and numeric handling.
- QA Trainer notes that functions like CHARINDEX and LIKE help filter text and that brackets and quotation marks are important for names and string literals.

Resources and practice environment

The trainer shares links and guidance for learners to access lab environments (GoToMyPC/LOD), session summaries, Microsoft Learn materials, and Microsoft SQL downloads for independent practice. These resources are recommended for learners who want hands-on practice creating databases, tables, and running queries beyond the classroom labs.

- QA Trainer posts links for GoToMyPC and LOD and instructs learners to sign in to their lab environments before starting exercises.
- QA Trainer points learners to the session summary transcript and notes available in the chat and recommends downloading the folder for later review.
- QA Trainer shares Microsoft Learn and Microsoft SQL resources for optional practice on creating databases, installing SQL Server, and running example scripts.

- QA Trainer recommends W3Schools SQL playground as an alternate practice environment preloaded with a Northwind-style dataset for extra practice.

Creating databases and tables

The trainer demonstrates creating a database and tables using CREATE DATABASE and CREATE TABLE statements, and explains column definitions and the importance of data types. The instructor emphasizes that full coverage of data types and schema design will come in a subsequent module but provides a quick live example and inserts sample rows for practice.

- QA Trainer demonstrates using CREATE DATABASE to set up a personal lab database and instructs learners to use the 'USE' statement to target the correct database.
- QA Trainer explains CREATE TABLE syntax, mentions column attributes and briefly refers to data types (INT, VARCHAR, MONEY) to be covered in a later module.
- QA Trainer inserts sample rows into created tables and verifies them with SELECT queries to show the create-and-query cycle.

Module five lab session

Learners work through module five exercises focused on grouping and aggregates with a timed lab approach; the trainer encourages consulting provided solutions to avoid getting stuck and to progress through increasingly complex scenarios. The class extends the timer and checks completion status while guiding learners on where to pick up the next tasks.

- QA Trainer sets a 15-minute timer (then extends by 5 minutes) for learners to complete module five exercises and instructs them to consult solutions if they get stuck.
- Learners report progress: some finished (David, Ewelina), some still working (several students), and the QA Trainer tracks completion via chat and raised hands.
- QA Trainer advises saving SQL queries in the documents folder for future reference because later labs build on earlier work.
- QA Trainer reminds learners the labs will get longer and encourages use of solutions if they are stuck to avoid unnecessary troubleshooting time.

Multi-table design motivation

The trainer introduces multi-table design through a practical surgery/hospital example to explain entities (patient, doctor, appointment, treatment, hospital/department) and why data is split across tables with primary and foreign keys. This sets the stage for joins and entity-relationship concepts in subsequent modules and demonstrates how joining tables supports real-world reporting needs.

- QA Trainer uses a surgery/hospital scenario to illustrate entities (Patient, Doctor, Appointment, Treatment, Hospital) and why data belongs across multiple tables.
- QA Trainer explains that each table stores its own attributes and primary keys while foreign keys (e.g., patient ID on appointment) link entities together for combined queries.
- QA Trainer emphasizes that joins allow creation of new result sets containing selected attributes from multiple tables to serve reporting or operational needs.

Join types overview

The trainer covers join concepts including INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN and explains their behavioral differences in terms of which rows are returned. The session clarifies that INNER JOIN returns only matching rows, LEFT/RIGHT JOIN preserve all rows from one side with NULLs where matches are missing, and aliasing helps simplify multi-table queries.

- QA Trainer states that JOIN defaults to INNER JOIN and that INNER JOIN returns rows where the ON clause is true between two tables.
- QA Trainer explains LEFT JOIN returns all rows from the left table with NULLs for non-matching right-side rows, and RIGHT JOIN returns all rows from the right table similarly.
- QA Trainer recommends using table aliases to shorten query text and make multi-table queries easier to read, as demonstrated with product/category examples.
- QA Trainer demonstrates how LEFT, RIGHT and FULL JOIN change the number and completeness of rows returned using a supplier–order example and emphasizes choosing join direction based on which entity (order or supplier) is the reporting focus.

Practical join examples

Using Northwind-style sample tables, the trainer demonstrates practical JOIN queries (product–category, customer–order, employee–car) to show how joins produce combined results and how choice of LEFT vs RIGHT changes output focus. The instructor demonstrates filtering, ordering, and using aliases to build consumable outputs like catalogs and order lists.

- QA Trainer demonstrates joining product and category tables to display product name alongside category name by matching category IDs.
- QA Trainer shows a LEFT JOIN of customers to orders to list all customers including those without orders, and then switches to RIGHT JOIN to list only customers with orders.
- QA Trainer uses the student/attendance and employee/car analogies to explain that LEFT vs RIGHT JOIN choice depends on which side (table) you want to preserve fully in results.
- QA Trainer walks through layered multi-joins (customer → order → employee) and explains how mixing LEFT and RIGHT joins can make 'order' the dominant table and eliminate NULLs in the result for that focus.

Union/except/intersect usage

The trainer explains set operations (UNION, UNION ALL, EXCEPT, INTERSECT) as alternatives to joins when combining similar datasets from two SELECT statements. The instructor covers how UNION de-duplicates by default, UNION ALL preserves duplicates, EXCEPT returns rows in the first set not in the second, and INTERSECT returns rows common to both, with examples combining product and order-detail derived sets.

- QA Trainer explains UNION merges two SELECT result sets with matching columns and that UNION removes duplicates while UNION ALL preserves them.

- QA Trainer demonstrates using GROUP BY/HAVING in one SELECT and then UNIONing with another SELECT that filters by price to produce combined product ID lists.
- QA Trainer shows EXCEPT to return rows that meet the first criteria but exclude rows from the second criteria, and INTERSECT to return only rows that meet both criteria.

Self-join patterns

The trainer introduces self-joins to relate rows within the same table, using an employee → manager example to map each employee to their manager by joining the employee table to itself. This demonstrates how foreign-key-like attributes that reference the same table enable hierarchical queries and reporting.

- QA Trainer demonstrates a self-join on the employee table by matching employee.report_to to manager.employee_id to derive manager names for each employee.
- QA Trainer points out that self-joins reveal hierarchical relationships and that not all rows will have a manager (managers can have NULL report_to).
- QA Trainer attributes observed manager values (employee ID two and five are managers) and shows concatenation to produce a single manager-name column.

NULL handling and formatting

The trainer and learners discuss strategies for replacing NULLs and formatting values for presentation, introducing ISNULL, CAST/CONVERT, and CASE/WHEN as practical techniques. They demonstrate nesting functions (ISNULL wrapped around CAST) to convert dates to text and provide fallback strings for NULL values, and preview a dedicated functions module.

- David asks about replacing NULLs in output and QA Trainer confirms the use of ISNULL (and similar functions) will be taught in the next module.
- QA Trainer demonstrates nesting ISNULL with CAST to convert a MIN(order_date) result to nvarchar and replace NULL with a text label, attributing the nested approach to a suggestion from Victoria.
- Victoria suggests using CASE WHEN as an alternative, and QA Trainer confirms both CASE WHEN and ISNULL can be applied in the SELECT clause for replacing NULLs.

Functions and data types overview

The trainer introduces scalar functions (text, date, NULL, and conversion functions) as tools to manipulate and format values in queries, and outlines data types (integer, char, varchar, nchar, nvarchar, datetime/time variants) to explain why casting/formatting matters. The instructor emphasizes choosing appropriate column types during design and shows how functions (CAST/CONVERT) let you change representation at query time.

- QA Trainer defines functions as stored SQL instructions returning scalar values and lists the function categories to be covered: text, date, NULL, and conversion functions.
- QA Trainer explains data-type choices (INT, CHAR, VARCHAR, NCHAR, NVARCHAR, various date/time types) and their implications for storage, validation, and formatting.

- QA Trainer recommends using NVARCHAR for multilingual data and describes CHAR vs VARCHAR trade-offs in fixed vs variable storage.
- QA Trainer connects functions to data types by showing how CAST/CONVERT allow formatting (for example converting a date to nvarchar) so other functions like ISNULL can replace NULLs with text.

Text and date function demos

The trainer demonstrates common text functions (LEFT, RIGHT, SUBSTRING, UPPER, LOWER) and describes date functions to extract date parts (year/month/day) and formatting options. Examples show how to control visible characters for privacy or reporting and how date functions provide targeted date components or formatted strings.

- QA Trainer demos LEFT/RIGHT to return fixed numbers of characters from strings and SUBSTRING to return a slice starting at a specified position, attributing use cases like partial name disclosure for privacy.
- QA Trainer demos UPPER and LOWER to normalize case for exports and shows concatenation to combine columns into a single display field.
- QA Trainer explains date functions extract parts (year/month/day) and that casting to text may be required to display date parts alongside strings.

Date/time functions detail

The trainer explains SQL Server date/time types and functions, demonstrating precision differences (datetime2, smalldatetime) and timezone-aware types for offset storage. He shows DATEPART/DATEDIFF usage to extract components or compute differences (days/months/years) and demonstrates GETDATE() for current timestamps and formatting via CAST/CONVERT with style codes for US/UK representations.

- QA Trainer explains datetime2 supports up to 100-nanosecond precision and smalldatetime stores only hours/minutes/seconds as context for choosing types.
- QA Trainer demonstrates DATEPART and DATEDIFF to extract or compute differences between order_date and shipped_date for SLA calculations.
- QA Trainer confirms GETDATE() returns the current PC/system timestamp and that CONVERT with format codes (e.g., 103, 106, 130) produces UK or full-month text formats as required.

Business day calculations question

Learners ask about excluding weekends and bank holidays from date difference calculations and the trainer acknowledges the platform has no single built-in function for business-day differences. He suggests custom logic or external calendars are typically required and volunteers to research possible functions or approaches for the group.

- Melinda asks how to exclude Saturdays, Sundays and bank holidays from day-difference calculations, prompting a trainer response.

- QA Trainer says the standard DATEDIFF counts calendar days and that excluding weekends/holidays likely requires custom code or a lookup/calendar table and commits to looking into options.
- QA Trainer notes Power BI has built-in approaches and acknowledges bank holidays add complexity requiring collected holiday data or specialized functions.

NULL functions and nesting

The trainer demonstrates ISNULL and NULLIF to replace or set values to NULL and shows nesting techniques to handle multiple cleanup rules at once. Examples include converting deprecated region codes to NULL and replacing NULLs with 'Unknown' for reporting exports, and the trainer shows how to wrap conversion/formatting inside NULL-handling functions.

- QA Trainer demonstrates ISNULL to replace NULL columns with a literal like 'Unknown' for cleaner exports.
- QA Trainer shows NULLIF to set specific values (e.g., deprecated region codes) to NULL so they can be uniformly handled afterwards.
- QA Trainer illustrates nesting convert/cast with ISNULL so a converted date string can be used as the NULL fallback expression.

CAST/CONVERT and try variants

The session covers CAST vs CONVERT semantics and the use of TRY_CAST/TRY_CONVERT for safer conversions; the trainer explains when to change data types versus when to change display formats. He shows converting dates to nvarchar with a format code, converting numeric types to money/integer, and advises using TRY_ variants to avoid hard failures when source strings don't match the target type.

- QA Trainer defines CAST as a type conversion and CONVERT as providing formatting options, especially for dates, and gives examples converting dates to nvarchar with specific format codes.
- QA Trainer demonstrates changing numeric types (e.g., money to integer) with CAST/CONVERT and explains the performance consideration of parsing strings.
- QA Trainer recommends TRY_CAST/TRY_CONVERT where source values may not conform to the target type so conversions fail gracefully instead of throwing errors.

Lab structure and script templates

The trainer explains that from module seven onward labs provide prewritten SQL script templates (QATSQL folder) that learners modify rather than write from scratch, reducing typing but increasing manipulation of existing scripts. He shows where to find the common files on GitHub and how to open/save them in the lab environment, and flags one common gotcha (use comments to hide UNION ALL sections for task one).

- QA Trainer instructs learners to copy or open the provided QATSQL scripts (common files → beginning extension) and then save a modified version as their working query.

- QA Trainer warns that module seven expects learners to edit provided script templates rather than author full scripts, which reduces typing but shifts emphasis to script manipulation.
- QA Trainer notes a specific task instruction to comment out sections (/* ... */) around UNION ALL for task one so the initial portion can be executed independently.

Lab troubleshooting and support

During the lab the trainer triages learner issues with convert/ISNULL tasks, joins into individual sessions (joining VMs) to debug scripts, and provides targeted fixes such as wrapping dates with CONVERT to nvarchar and supplying the correct format code. The trainer confirms many learners progress while offering to review and join machines for those still stuck.

- Julia reports errors on exercise three tasks; QA Trainer debugs with her and explains the need to CONVERT the date to nvarchar with a UK format code before applying ISNULL.
- QA Trainer volunteers to join individual learner VMs to troubleshoot scripts and confirms several learners (Kim, David, Chris) report success after applying suggested fixes.
- QA Trainer points out a lab-specific ambiguous instruction (separating first/last name) that commonly causes failures and advises checking statement six when task output is missing.

File transfer and lab access guidance

Learners ask how to persist or download work from browser-based LOD labs; the trainer recommends using Google Drive/OneDrive or GitHub to transfer files and explains GoToMyPC provides native file transfer but LOD requires upload to cloud storage. He also explains lab retention windows and alternative practice options if access is lost.

- Melinda asks how to retrieve files from the LOD VM and QA Trainer recommends saving work to OneDrive/Google Drive or copying scripts from GitHub into the VM.
- QA Trainer explains GoToMyPC has file transfer capabilities while LOD access is browser-based and requires cloud storage for file movement.
- QA Trainer advises GoToMyPC users they may lose lab access sooner than LOD users and suggests W3Schools or the 'Run your first SQL' hands-on labs as alternatives.

Views: purpose and creation

The trainer introduces views as reusable, template-like SELECT results stored in the database to simplify repeated complex queries and to enforce access control. He demonstrates CREATE VIEW (without ORDER BY) to save a join result (product by category), shows where views appear in the database explorer, and highlights that users can select from a view like a table and filter or order on retrieval.

- QA Trainer explains views store complex SELECT logic so you can run SELECT from view_name later instead of rewriting joins and filters.
- QA Trainer demonstrates CREATE VIEW and explains ORDER BY must be applied when selecting from the view, not inside the view definition itself.

- QA Trainer points out views appear under Tables → Views in the database explorer and can simplify permissions by granting access to views instead of base tables.

Views maintenance and transparency

The trainer shows learners how to inspect and modify view definitions so they can see the original SELECT logic and adjust filters (for example, discontinued flags) when requirements change. He demonstrates editing a view to change criteria and re-saving it so subsequent selects reflect the new logic.

- Melinda asks how to see the logic behind a view and QA Trainer shows how to script or open the view definition to review and update the original SQL.
- QA Trainer confirms you can change a view's WHERE clause (for example the discontinued flag) and save the updated view so future queries use the revised logic.
- QA Trainer demonstrates selecting only required columns from a view to produce tailored outputs without changing the view itself.

Stored procedures fundamentals

The trainer contrasts views with stored procedures and demonstrates creating, storing, and executing parameterized stored procedures as programmable code that can return records or modify data. He shows how procedures appear under Programmability → Stored Procedures, explains parameter usage and required arguments at execution, and provides examples for report-style procedures (customers by city, premium products).

- QA Trainer explains stored procedures are programmable SQL code stored in the database that you execute with the EXECUTE command.
- QA Trainer demonstrates creating a parameterized procedure (e.g., customers by city) and notes that callers must supply required parameters when executing the procedure, as highlighted in a learner Q&A with Melinda.
- QA Trainer shows where stored procedures appear in the database explorer under Programmability → Stored Procedures and how to recall and execute them by name.
- QA Trainer stores a premium-products procedure example and explains the procedure can encapsulate complex logic (including UNIONs and GROUP BY) so users only need to execute the procedure to get results.

Table types and functions

The trainer differentiates table-like objects—views, table-valued functions, derived tables, temporary tables—and explains their use cases, lifecycle, and when to pick each for repeatable logic or session-scoped datasets. He also introduces more advanced grouping constructs (ROLLUP, CUBE) for multi-level aggregations and explains transaction semantics at a high level to close the module.

- QA Trainer explains table-valued functions return a table result when executed and that inline table-valued functions differ from multi-statement functions in declared-variable scope.

- QA Trainer describes derived tables as ad-hoc, query-scoped table results created inside a single SELECT and contrasts them with stored objects like views and procedures, as explained by the QA Trainer.
- QA Trainer cautions that temporary tables persist only for the session (and until all sessions referencing them disconnect) and recommends them for repeated, transient datasets, as noted by the QA Trainer.
- QA Trainer introduces ROLLUP as an advanced GROUP BY extension that generates hierarchical subtotal rows and attributes the NULL group_level rows to summarisation totals.
- QA Trainer explains CUBE extends ROLLUP to produce all combinations of grouping levels and that both are useful for quarter/category profit-and-loss summaries used by reporting apps.
- QA Trainer defines transaction behavior as 'complete' or 'not complete' and explains that incomplete transactions leave no recorded changes, providing basic ACID-style context for learners.

Session close and certification guidance

The trainer wraps the module by confirming views, stored procedures, table types, ROLLUP/CUBE and transactions are the final key topics and gives instructions for submitting reflection statements and downloading the certificate of completion from the QA learning portal. He remains available for learners to continue labs independently and offers support for lab access issues and next-module pre-learning setup.

- QA Trainer instructs learners to mark pre-learning and module content complete on the QA learning portal to unlock and download the certificate of completion.
- QA Trainer offers to adjust individual learner records (e.g., set module dates) so the practice portal sections appear, as he did live for Julia.
- QA Trainer advises learners to upload the certificate and a reflection statement to the QA portal for module sign-off and notes screenshots are optional.
- QA Trainer reminds learners that module nine–eleven labs are more complex and recommends using provided solutions when attempting them independently.